6. Strings, Runes, Bytes, and UTF-8

06. Strings, Runes, Bytes, and UTF-8

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `06-strings-runes-bytes`

Learning objectives

- Explain that Go strings are immutable byte sequences (often UTF-8)
- Iterate by rune vs by byte correctly
- Convert between string, []byte, and []rune consciously
- Avoid indexing strings when Unicode matters

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Real text is UTF-8. Treating `len(s)` as character count breaks emoji, names, and logs. Runes vs bytes is mandatory for correct processing.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- string is an immutable byte sequence, usually UTF-8.
- `range` over string yields runes; indexing `s[i]` yields a byte.
- []byte ↔ string conversions copy; hot paths may stay in bytes.
- strings.Builder avoids quadratic `+` concatenation in loops.
- A rune is int32 holding a Unicode code point — not always one user-perceived character.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-strings, && cd /tmp/gw-strings,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func main() {
    s := "Go👍"
    fmt.Println("bytes", len(s))
    n := 0
    for _, r := range s {
        n++
        fmt.Printf("%q ", r)
    }
    fmt.Println("\nrunes", n)
}
```

Run it:

```
go run .
```

Expected output:

```
bytes 6
'G' 'o' '👍'
runes 3
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Compare len vs rune count for emoji text.
2. Reverse by runes and verify round-trip.
3. Intentionally slice mid-code-point and observe invalid UTF-8.
4. Build a string with strings.Builder in a loop of 1000.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Strings are bytes

```
s := "Go👍"
fmt.Println(len(s)) // byte length, not character count
for i, r := range s {
    // r is rune (code point); i is byte index
    fmt.Println(i, string(r))
}
```

A **rune** is an alias for `int32` representing a Unicode code point. UTF-8 encodes code points in 1–4 bytes. Indexing `s[i]` gives a **byte**, not a character.

Conversions

```
b := []byte("hi")
s := string(b)
rs := []rune("naïve")
```

Conversions copy data. Hot paths sometimes stay in `[]byte` to avoid allocations.

Building strings

Prefer `strings.Builder` when concatenating in a loop — repeated `+` allocates often.

Failure modes to watch while coding

- Using `len(s)` as character count for Unicode text
- Slicing mid-code-point and creating invalid UTF-8
- Comparing strings with locale rules (Go compares bytes)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python 3 str is Unicode code points; Go string is bytes.
- s[i] in Go is a byte; in Python it is a 1-char str.
- len(s) in Python counts chars; len(s) in Go counts bytes.
- Encoding issues appear at I/O — be explicit at boundaries.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

For grapheme clusters (user-perceived characters) you need extra libraries; runes are the first correct step beyond bytes.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Strings, Runes, Bytes, and UTF-8” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Using `len(s)` as character count for Unicode text
- Slicing mid-code-point and creating invalid UTF-8
- Comparing strings with locale rules (Go compares bytes)

Exercises

Work in `exercises/06-strings-runes-bytes/`. Read that folder's `README.md` before you code.

What you will build

Implement RuneCount and ReverseRunes that respect Unicode code points.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/06-strings-runes-bytes/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Difference between `for i := range s` and `for i, r := range s`?
2. Why is `string` immutable important for concurrency?
3. When use `[]byte` instead of `string`?

Further reading

- https://go.dev/blog/strings
- https://go.dev/blog/normalization